PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Gabriel Lobo de Sant'Anna Rosas

PROJETO FINAL DE GRADUAÇÃO

CENTRO TÉCNICO CIENTÍFICO - CTC

DEPARTAMENTO DE INFORMÁTICA

Curso de Graduação em Engenharia de Computação

Rio de Janeiro, Novembro de 2026

Pontifícia Universidade Católica
do Rio de Janeiro



Gabriel Lobo de Sant'Anna Rosas

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Relatório de Projeto Final, apresentado ao programa de Graduação do Departamento de Informática da PUC-Rio como requisito parcial para a obtenção do título de Engenheiro de Computação.

Orientador: Adriano Francisco Branco
Co-orientador: Jonatas dos Santos Grosman

Rio de Janeiro
Novembro de 2026

Resumo

Lobo de Sant'Anna Rosas, Gabriel. Francisco Branco, Adriano. dos Santos Grosman, Jonatas. **Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais**. Rio de Janeiro, 2026. 37 p. Relatório de Projeto Final – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Provedores de nuvem precisam decidir continuamente quantas instâncias manter ativas: recursos em excesso elevam o custo, e recursos insuficientes violam o acordo de nível de serviço. A alocação reativa só ajusta a capacidade depois de observar a mudança na demanda e, por isso, chega atrasada nos picos. Este trabalho investiga o quanto a qualidade da previsão de carga reduz o custo de alocação sem aumentar as violações de serviço. Para atribuir as diferenças ao previsor, a política de alocação é mantida fixa e apenas o previsor muda. São comparados métodos tradicionais de previsão (ingênuo, média móvel, sazonal ingênuo, Holt-Winters e ARIMA) e uma rede neural recorrente do tipo GRU, além de uma política reativa e do dimensionamento estático pelo pico do treino, sobre séries de utilização agregadas a partir do rastro público de um *cluster* da Alibaba e avaliadas em um centro de dados simulado. As métricas são o custo normalizado pelo de um oráculo e a fração de intervalos com violação. Os resultados preliminares mostram que a previsão reduz muito o custo frente ao dimensionamento estático e de forma modesta frente à política reativa, e que a GRU é competitiva com o ARIMA, sem vantagem clara. Esta etapa também define o ambiente e o plano do Projeto Final II.

Palavras-chave

Computação em nuvem; Alocação de recursos; Previsão de carga; Redes neurais; Acordo de nível de serviço.

Abstract

Lobo de Sant'Anna Rosas, Gabriel. Francisco Branco, Adriano (Advisor). dos Santos Grosman, Jonatas (Co-advisor). **Predictive cloud resource allocation via workload forecasting with neural networks for operating cost minimization**. Rio de Janeiro, 2026. 37 p. Undergraduate Thesis – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Cloud providers must continuously decide how many instances to keep active: excess resources raise costs, while insufficient resources violate the service level agreement. Reactive allocation adjusts capacity only after observing a change in demand and therefore lags behind peaks. This work investigates how much the quality of workload forecasting reduces allocation cost without increasing service violations. To attribute differences to the forecaster, the allocation policy is kept fixed and only the forecaster changes. Traditional forecasting methods (naive, moving average, seasonal naive, Holt-Winters and ARIMA) and a recurrent neural network of the GRU type are compared, together with a reactive policy and static sizing at the training peak, using utilization series aggregated from a public Alibaba cluster trace and evaluated in a simulated data center. The metrics are the cost normalized by that of an oracle and the fraction of intervals with violations. Preliminary results show that forecasting greatly reduces cost compared with static sizing and only modestly compared with the reactive policy, and that the GRU is competitive with ARIMA, with no clear advantage. This stage also defines the environment and the plan for the second part of the project.

Keywords

Cloud computing; Resource allocation; Workload forecasting; Neural networks; Service level agreement.

Sumário

1 Introdução **9**
1.1 Motivação 9
1.2 Definição do problema 9
1.3 Justificativa 10
1.4 Escopo e delimitação 10
1.5 Organização do documento 11

2 Situação Atual **12**
2.1 Conceitos e terminologia 12
2.2 Abordagens para previsão de carga 14
2.3 Abordagens para alocação de recursos 16
2.4 Comparativo dos trabalhos relacionados e lacuna 18
2.5 Tecnologias utilizadas 19

3 Objetivos do trabalho **20**
3.1 Solução proposta 20
3.2 Objetivos 21
3.3 Escopo do sistema 23
3.4 Usuários e situações apoiadas 23
3.5 Avanços em relação às soluções existentes 24
3.6 Itens a serem implementados e apresentados à banca 24

4 Plano de ação **26**
4.1 Plano previsto 26
4.2 Atividades realizadas 27
4.3 Mudanças em relação ao plano 28
4.4 Dificuldades encontradas 29
4.5 Resultados preliminares 30

5 Cronogramas **33**
5.1 Projeto Final I 33
5.2 Projeto Final II 33

Referências **35**

Lista de figuras

Figura 3.1 Arquitetura da solução proposta. 20

Figura 4.1 Demanda de processador do *cluster* nos oito dias do rastro, em intervalos de cinco minutos, com a divisão cronológica. O trecho tracejado é a lacuna de coleta preenchida. O painel inferior mostra a cobertura, isto é, a fração das M máquinas que enviaram medições em cada intervalo; abaixo da linha pontilhada, o intervalo é tratado como lacuna. 27

Figura 4.2 Custo normalizado pelo do oráculo contra a fração de intervalos com violação no conjunto de teste, com $L = 1$. Cada curva varia o parâmetro de uma política (m na preditiva e u^* na reativa), e as linhas verticais marcam $\varepsilon = 1\%$ e 5%. O eixo horizontal é linear até 1% e logarítmico a partir daí. Na legenda, *naive* designa o método ingênuo. 31

Lista de tabelas

Tabela 2.1 Comparativo entre os trabalhos relacionados e a proposta deste trabalho. O traço indica que o critério não se aplica. 18
Tabela 2.2 Ferramentas utilizadas em cada etapa do *pipeline*. 19

Tabela 3.1 Parâmetros do protocolo experimental adotados no protótipo. 23
Tabela 3.2 Itens a serem implementados e situação ao fim do Projeto Final I. 24

Tabela 4.1 Fases previstas para a construção do protótipo no Projeto Final I e datas de conclusão, em 2026. 26
Tabela 4.2 Erro das previsões no conjunto de teste com horizonte $h = 1$, em 462 origens. RMSE, MAE e viés estão em máquinas equivalentes; viés positivo indica superestimação. 30
Tabela 4.3 Pontos de operação no conjunto de teste, com $L = 1$. O parâmetro de cada política foi escolhido na validação (Equação 3-1). Valores de V em negrito ultrapassam ε no teste. 31

Tabela 5.1 Cronograma do Projeto Final I, em 2026. 33
Tabela 5.2 Cronograma proposto para o Projeto Final II. 34

Lista de Abreviaturas

AIC – Critério de informação de Akaike (*Akaike Information Criterion*)

ARIMA – Modelo autorregressivo integrado de médias móveis (*AutoRegressive Integrated Moving Average*)

CPU – Unidade central de processamento (*Central Processing Unit*)

GPU – Unidade de processamento gráfico (*Graphics Processing Unit*)

GRU – *Gated Recurrent Unit*

HBNN – Rede neural bayesiana (*Bayesian Neural Network*), na nomenclatura de Rossi et al.

HPA – *Horizontal Pod Autoscaler*

LSTM – *Long Short-Term Memory*

LSTMD – LSTM com saída distribucional (*LSTM Distributional*)

MAE – Erro absoluto médio (*Mean Absolute Error*)

MAPE – Erro percentual absoluto médio (*Mean Absolute Percentage Error*)

MSE – Erro quadrático médio (*Mean Squared Error*)

OE – Objetivo específico

PF – Projeto Final

QoS – Qualidade de serviço (*Quality of Service*)

QP – Questão de pesquisa

RMSE – Raiz do erro quadrático médio (*Root Mean Squared Error*)

RNN – Rede neural recorrente (*Recurrent Neural Network*)

SLA – Acordo de nível de serviço (*Service Level Agreement*)

SLI – Indicador de nível de serviço (*Service Level Indicator*)

SLO – Objetivo de nível de serviço (*Service Level Objective*)

SQL – *Structured Query Language*

1
Introdução

1.1
Motivação

A computação em nuvem consolidou-se como a principal forma de disponibilizar infraestrutura computacional para empresas e instituições. Nesse modelo, o provedor mantém grandes centros de dados e precisa decidir, de forma contínua, quanto de processamento, memória e armazenamento deve estar disponível para atender à demanda de seus clientes. Como a demanda tende a crescer, impulsionada sobretudo por aplicações de Inteligência Artificial e de análise de grandes volumes de dados, a gestão eficiente desses recursos tornou-se um fator central para a sustentabilidade econômica dos provedores (ROSSI et al., 2025).

A dificuldade está no fato de que a carga de trabalho não é constante. Ela apresenta picos, padrões periódicos e variações aparentemente aleatórias. Diante disso, a abordagem mais comum é a alocação reativa, na qual o sistema ajusta os recursos apenas depois de observar uma mudança na demanda, normalmente por meio de regras de limiar sobre a utilização. Como a reação ocorre com atraso, esse tipo de estratégia tende a degradar o desempenho durante os picos e a desperdiçar recursos nos períodos de baixa utilização (BODRA; KHAIRNAR, 2025).

Esse cenário cria um dilema permanente. Manter recursos em excesso (superprovisionamento) garante a qualidade do serviço, mas eleva o custo operacional. Manter recursos de menos (subprovisionamento) reduz o custo, mas provoca violações dos Acordos de Nível de Serviço (*Service Level Agreements*, SLA) firmados com os clientes. Uma alternativa é a alocação preditiva, que utiliza o histórico de uso para antecipar a demanda futura e provisionar recursos antes que eles sejam necessários (BODRA; KHAIRNAR, 2025). Nesse contexto, prever a carga com precisão passa a ser uma pré-condição para otimizar a alocação (ROSSI et al., 2025).

A previsão de carga em nuvem foi tradicionalmente tratada com modelos estatísticos lineares, como o modelo autorregressivo integrado de médias móveis (ARIMA) (CALHEIROS et al., 2015). Mais recentemente, redes neurais profundas, em especial as arquiteturas recorrentes, passaram a ser aplicadas ao problema com bons resultados, inclusive em abordagens multivariadas que consideram simultaneamente o uso de processador e de memória (XU et al., 2022).

1.2
Definição do problema

O problema tratado neste trabalho tem duas partes encadeadas. A primeira é um problema de previsão: dado o histórico de utilização de recursos de um *cluster*, amostrado em intervalos regulares, estimar a demanda de processador e

de memória em um horizonte curto. A segunda é um problema de decisão: a partir dessa estimativa, determinar quantas instâncias de recursos (máquinas virtuais ou contêineres) devem permanecer ativas em cada intervalo.

De forma preliminar, a decisão pode ser formulada como a minimização do custo operacional sujeita a uma restrição de qualidade de serviço:

$$\min_{n_1,\ldots,n_T} \sum_{t=1}^{T} c \cdot n_t \quad \text{sujeito a} \quad \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \leq \varepsilon, \tag{1-1}$$

em que n_t é o número de instâncias ativas no intervalo t, c é o custo de uma instância por intervalo, k é a capacidade de uma instância, d_t é a demanda real observada, $\mathbf{1}[\cdot]$ é a função indicadora e ε é a fração máxima de intervalos com violação de SLA tolerada. Como d_t não é conhecida no momento da decisão, a escolha de n_t depende da previsão $\hat{d}_t$, e a qualidade dessa previsão afeta diretamente o custo e o número de violações. Essa formulação é refinada no Capítulo 3, que define como a restrição é verificada na prática (Equação 3-1).

1.3 Justificativa

A relevância do problema é econômica e operacional. Cada ponto percentual de recursos ociosos representa custo para o provedor, e cada violação de SLA representa perda de qualidade percebida pelo cliente e, eventualmente, penalidades contratuais. Apesar do volume de trabalhos sobre previsão de carga, falta uma avaliação de ponta a ponta, com dados públicos, que meça quanto a previsão reduz o custo de alocação quando redes neurais e métodos tradicionais são submetidos à mesma política, lacuna discutida na Seção 2.4.

1.4 Escopo e delimitação

Incluído. Rastros públicos da Alibaba (Alibaba Group, 2018) e do Google (WILKES, 2020), agregados em séries de uso de processador e de memória; horizontes de previsão curtos, na escala de minutos; alocação em um centro de dados simulado, com custo por instância ativa e violação de SLA nos intervalos em que a demanda excede a capacidade; e comparação entre uma rede neural e métodos tradicionais de previsão sob a mesma política de alocação, ao lado das políticas de referência da Seção 3.1.

Fora do escopo. Implantação em uma nuvem real, posicionamento e migração de máquinas virtuais entre servidores físicos, consumo de energia, múltiplos provedores e aprendizado por reforço para a decisão. Na simulação, o posicionamento segue a política padrão do simulador, a mesma para todos os previsores.

1.5 Organização do documento

O restante deste documento está organizado da seguinte forma. O Capítulo 2 apresenta a terminologia básica e as principais abordagens existentes para previsão de carga e alocação de recursos em nuvem, com suas limitações, além das tecnologias utilizadas. O Capítulo 3 descreve a solução proposta, seus objetivos específicos e o que será efetivamente implementado. O Capítulo 4 revisa o plano de ação, comparando as atividades previstas com as realizadas, e apresenta os resultados preliminares do protótipo. Por fim, o Capítulo 5 apresenta os cronogramas do Projeto Final I (PF I) e do Projeto Final II (PF II).

2
Situação Atual

Este capítulo define a terminologia (Seção 2.1), descreve as abordagens existentes para previsão de carga e para alocação de recursos, com suas limitações (Seções 2.2 e 2.3), compara os trabalhos mais próximos desta proposta (Seção 2.4) e apresenta as tecnologias utilizadas (Seção 2.5).

2.1
Conceitos e terminologia

2.1.1
Elasticidade, carga de trabalho e séries temporais

Uma das características essenciais da computação em nuvem é a elasticidade rápida, isto é, a capacidade de provisionar e liberar recursos, em alguns casos de forma automática, acompanhando a demanda (MELL; GRANCE, 2011). A carga de trabalho (*workload*) de um *cluster*, formada por tarefas em lote, serviços de longa duração e requisições de usuários, se manifesta como utilização de recursos, principalmente processador e memória. Agregada em intervalos regulares de duração Δ, ela forma uma série temporal $y_1, \ldots, y_t$ que combina tendência, padrões periódicos, ruído e picos abruptos (XU et al., 2022). Prever a carga significa estimar $\hat{y}_{t+h}$, o valor da série h intervalos à frente (horizonte de previsão), a partir das w observações mais recentes (janela de entrada). O intervalo em que a previsão é feita é a origem τ: uma previsão feita em τ só pode usar observações até τ. A previsão é univariada quando usa apenas a própria série e multivariada quando incorpora outras, como a memória na previsão do processador (XU et al., 2022).

Entre os rastros (*traces*) públicos de *clusters* reais estão o do Google, com oito células do gerenciador Borg em maio de 2019 (WILKES, 2020), e o da Alibaba de 2018, com cerca de 4.000 máquinas ao longo de oito dias (Alibaba Group, 2018); transformá-los em séries regulares faz parte deste trabalho. Intervalos de cinco minutos são os usuais com esses rastros (ROSSI et al., 2025; XU et al., 2022); Rossi et al. (2025) preveem a demanda dez minutos à frente, horizonte que consideram suficiente para a maioria das aplicações de alocação. O intervalo deve comportar a inicialização de uma instância (CALHEIROS et al., 2015).

2.1.2
Acordos de nível de serviço

A qualidade de um serviço em nuvem é expressa pelo indicador de nível de serviço (*Service Level Indicator*, SLI), uma medida quantitativa como a latência, pelo objetivo de nível de serviço (*Service Level Objective*, SLO), um valor-alvo para um SLI, e pelo SLA, que formaliza esses objetivos em contrato e define as

consequências, geralmente financeiras, de não cumpri-los (BEYER et al., 2016). Como os rastros públicos não registram a latência percebida pelos usuários, a literatura de alocação aproxima a violação de SLA por medidas derivadas da utilização, como a fração do tempo em que os servidores operam com utilização máxima (BELOGLAZOV; BUYYA, 2012). Este trabalho adota uma aproximação do mesmo tipo: o SLI é medido por intervalo, atendido quando $d_t \leq k \cdot n_t$ na notação da Equação 1-1, e o SLO é que no máximo uma fração ε dos intervalos fique sem atendimento, que é a restrição daquela equação. A simplificação não captura filas nem a degradação gradual perto da saturação, mas permite comparar estratégias de forma consistente.

2.1.3 Provisionamento reativo e preditivo

O escalonamento automático (*autoscaling*) ajusta os recursos de uma aplicação sem intervenção humana; este trabalho trata do escalonamento horizontal, que altera o número de instâncias. Entre as famílias de técnicas classificadas por Lorido-Botran, Miguel-Alonso e Lozano (2014), a distinção central aqui é entre o provisionamento reativo, que decide a partir do que já foi observado, normalmente a utilização corrente, e o preditivo, que decide a partir de uma estimativa $\hat{d}_{t+h}$ da demanda futura ($y_t = d_t$) e já é oferecido por provedores comerciais (Amazon Web Services, 2024).

Como uma nova instância leva um tempo para ficar disponível, o atraso de inicialização (MAO; HUMPHREY, 2012), a decisão tomada com as observações até o intervalo τ só passa a valer em $\tau + L$, em que L é a antecedência da decisão (*lead time*), igual a um intervalo mais o atraso de inicialização; sem atraso, $L = 1$. A política reativa decide em τ com a utilização observada em τ, e a preditiva tenta antecipar a demanda de $\tau + L$. Em cada intervalo, a decisão pode errar em duas direções: com $k \cdot n_t > d_t$, a capacidade ociosa $k \cdot n_t - d_t$ é custo sem retorno (superprovisionamento); com $k \cdot n_t < d_t$, parte da demanda não é atendida (subprovisionamento).

2.1.4 Métricas de avaliação

Métricas de previsão. Medem o quanto a previsão se aproxima do valor real, independentemente de como ela será usada. Para N previsões $\hat{y}_i$ e valores reais y_i, as mais comuns são o erro quadrático médio (MSE), sua raiz (RMSE), o erro absoluto médio (MAE) e o erro percentual absoluto médio (MAPE), às quais este trabalho acrescenta o viés, que é o erro médio com sinal:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad \mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|,$$
$$\mathrm{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|, \qquad \text{viés} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i). \tag{2-1}$$

Como o MAPE é instável com y_i próximo de zero, ele é calculado só sobre os demais pontos, e a fração mantida (cobertura da máscara) é informada. As quatro primeiras métricas são simétricas, mas, na alocação, a superestimação gera custo e a subestimação gera violações, de modo que um modelo com menor RMSE não leva necessariamente às melhores decisões. Por isso são usados também o viés, cujo valor negativo indica tendência a subestimar a demanda, e a taxa de subestimação, que é a fração das previsões abaixo do valor real.

Métricas de alocação. Medem o resultado das decisões tomadas a partir da previsão. Para T intervalos:

$$C = \sum_{t=1}^{T} c \cdot n_t, \qquad V = \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}\left[\, d_t > k \cdot n_t \,\right],$$
$$U = \frac{\sum_t \max(0,\, d_t - k \cdot n_t)}{\sum_t d_t}, \qquad O = \frac{\sum_t \max(0,\, k \cdot n_t - d_t)}{\sum_t k \cdot n_t},$$
$$A = \sum_{t=2}^{T}\mathbf{1}\left[\, n_t \neq n_{t-1} \,\right], \qquad C_{\mathrm{norm}} = \frac{C}{C_{\text{oráculo}}}, \tag{2-2}$$

em que C é o custo total, V é a fração de intervalos com violação, U é a fração da demanda não atendida, O é a fração da capacidade ociosa e A é o número de alterações em n_t, indesejáveis na prática. [PENDENTE: U e O: reportar na Tabela 4.3 ou remover as definições]. C e V são, respectivamente, a função objetivo e o lado esquerdo da restrição da Equação 1-1. O custo é normalizado pelo de um oráculo, que conhece a demanda futura e aloca, em cada intervalo, o menor número de instâncias que a atende, $\lceil d_t/k \rceil$. Como nenhuma alocação mais barata atende toda a demanda, $C_{\text{oráculo}}$ é o piso do custo sem violações; uma política que viola pode custar menos que o oráculo, e por isso C_{norm} deve ser lido sempre junto com V.

2.2 Abordagens para previsão de carga

2.2.1 Modelos estatísticos

Os métodos mais simples de previsão servem como referência para todos os demais (HYNDMAN; ATHANASOPOULOS, 2018). O método ingênuo, ou de persistência, assume que o próximo valor será igual ao último observado, $\hat{y}_{t+1} = y_t$. A média móvel usa a média das w últimas observações. O sazonal ingênuo repete

o valor observado um período sazonal antes, $\hat{y}_{t+h} = y_{t+h-s}$, em que s é o período da sazonalidade, medido em intervalos, e $h \leq s$. O método de Holt-Winters acrescenta à suavização exponencial componentes de tendência e de sazonalidade, o que o torna adequado aos ciclos diários comuns em cargas de nuvem (HYNDMAN; ATHANASOPOULOS, 2018). A família ARIMA (*AutoRegressive Integrated Moving Average*), sistematizada pela metodologia de Box e Jenkins (BOX et al., 2015), aplica d diferenciações para tornar a série estacionária e descreve o resultado como combinação linear de p valores passados e de q erros de previsão passados; as ordens p, d e q podem ser escolhidas pela análise das funções de autocorrelação ou por critérios de informação, como o critério de informação de Akaike (AIC) (HYNDMAN; ATHANASOPOULOS, 2018).

Calheiros et al. (2015) aplicaram um ARIMA à demanda horária de requisições à Wikipédia e avaliaram no CloudSim (CALHEIROS et al., 2011) o efeito sobre a qualidade de serviço (QoS): provisionar pelos limites superiores dos intervalos de confiança, em vez do valor previsto, reduziu as violações de QoS ao custo de mais horas de máquina virtual, um ajuste aplicado depois da previsão que controla o equilíbrio entre custo e violações. A avaliação, contudo, envolve um único previsor.

Limitações. Os modelos estatísticos assumem relações lineares e, no ARIMA, estacionariedade após a diferenciação, hipóteses que as mudanças de regime e os picos das cargas de nuvem violam, e, em sua forma básica, são univariados. Por outro lado, exigem poucos dados, são baratos de ajustar e interpretáveis, e por isso devem estar em qualquer comparação: uma rede neural só se justifica se os superar sob as mesmas condições.

2.2.2 Redes neurais recorrentes

Para aplicar uma rede neural, a série é transformada em um problema supervisionado por janela deslizante, com as w observações mais recentes na entrada; para vários horizontes, a estratégia direta usa uma saída por horizonte, em vez de realimentar as próprias previsões e acumular erro (TAIEB et al., 2012). As redes neurais recorrentes (RNN) leem a janela em ordem e mantêm um estado interno que funciona como memória, mas têm dificuldade de aprender relações entre observações distantes (BENGIO; SIMARD; FRASCONI, 1994). A LSTM (*Long Short-Term Memory*) resolve esse problema com portas que aprendem o que guardar e o que esquecer (HOCHREITER; SCHMIDHUBER, 1997). A GRU (*Gated Recurrent Unit*) simplifica essa ideia com duas portas (CHO et al., 2014), sendo mais barata de treinar e com desempenho comparável (CHUNG et al., 2014). O esDNN (XU et al., 2022) combina uma GRU modificada com camadas convolucionais para prever processador e memória nos rastros da Alibaba e do Google; a GRU pura é competitiva em horizontes curtos, mas a avaliação usa apenas métricas de erro simétricas.

Limitações. Redes recorrentes exigem mais dados e mais ajuste do que os modelos estatísticos e, treinadas para minimizar um erro médio, tendem a suavizar os picos de demanda. Agulló López et al. (2025) mostram que previsores tradicionais e profundos falham justamente nesses picos, sem que as métricas usuais revelem a falha, e que ajustar a saída da previsão melhora a antecipação dos picos ao custo de mais superprovisionamento. Isso reforça que o erro médio não basta para avaliar um previsor usado na alocação.

2.2.3 Modelos com quantificação de incerteza

Rossi et al. (2025) propõem uma rede neural bayesiana (HBNN) e uma LSTM com saída distribucional (LSTMD), que preveem uma distribuição para a demanda de processador e de memória nos rastros do Google e da Alibaba, e mostram que métricas de erro pontual não distinguem bem os modelos, ao contrário de métricas de nível de serviço calculadas sobre o limite superior do intervalo de confiança: a taxa de sucesso, fração dos intervalos em que a demanda fica abaixo desse limite, e o total de recursos previstos, soma dos limites em relação à demanda. Os autores também concluem que a transferência de aprendizado funciona dentro de um mesmo provedor, mas se degrada entre provedores diferentes.

Limitações. Essas métricas são calculadas sobre a própria previsão, sem mecanismo de alocação, integração que os autores deixam como trabalho futuro, e supõem capacidade contínua, e não instâncias discretas com atraso de inicialização. Este trabalho mede diretamente as métricas de alocação e mantém a ideia de provisionar pelo limite superior na forma determinística da margem m (Seção 2.3.2).

2.3 Abordagens para alocação de recursos

2.3.1 Heurísticas e políticas reativas

As políticas baseadas em limiares são a forma mais difundida de escalonamento automático (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014). Se a utilização média das instâncias permanece acima de um limiar superior por alguns intervalos, uma instância é adicionada, e, se permanece abaixo de um limiar inferior, uma é removida. Um período de espera (*cooldown*) após cada ação impede oscilações. O *Horizontal Pod Autoscaler* (HPA) do Kubernetes usa uma variação proporcional, que calcula o número de réplicas pela razão entre a métrica observada u_t e um valor-alvo u^* (The Kubernetes Authors, 2024):

$$n_{t+1} = \left\lceil n_t \cdot \frac{u_t}{u^*} \right\rceil , \tag{2-3}$$

com uma faixa de tolerância, dentro da qual nenhuma ação é tomada, e uma janela de estabilização para reduções. O esDNN usa como referência outra política

reativa, que mantém ativa a média do número de máquinas dos cinco intervalos anteriores (XU et al., 2022).

Limitações. Por construção, as políticas reativas só respondem depois da mudança de carga, defasagem que o atraso de inicialização aumenta. Os limiares precisam ser ajustados para cada aplicação: limiares agressivos causam oscilações, e conservadores mantêm capacidade ociosa. Ainda assim, por serem simples, previsíveis e amplamente usadas, a variante proporcional do HPA (Equação 2-3) é a referência reativa deste trabalho.

2.3.2 Alocação guiada por previsão

Na alocação guiada por previsão, a decisão tem duas etapas desacopladas: um modelo estima a demanda $\hat{d}_{t+h}$ do intervalo em que a decisão terá efeito, e uma regra converte essa estimativa em número de instâncias, normalmente dividindo-a pela capacidade de uma instância e acrescentando uma margem de segurança $m \geq 0$, com um piso de $n_{\min}$ instâncias:

$$n_{t+h} = \max\left(n_{\min}, \left\lceil \frac{(1+m)\,\hat{d}_{t+h}}{k} \right\rceil\right). \tag{2-4}$$

Na alocação, t é a origem τ e o horizonte é a antecedência, $h = L$, para que a previsão se refira ao intervalo em que a decisão vale (Seção 2.1.3). Margens maiores reduzem V e aumentam C, e escolher a margem em um conjunto de validação é a forma prática de buscar o menor custo que respeita o limite ε da Equação 1-1. Com um modelo que fornece uma distribuição, a margem pode dar lugar a um quantil (ROSSI et al., 2025; CALHEIROS et al., 2015).

O MagicScaler (PAN et al., 2023) combina uma previsão com incerteza a uma otimização estocástica que minimiza o custo sujeito a um limite de demanda não atendida e, em *clusters* da Alibaba Cloud, reduziu a perda de qualidade de serviço em relação a uma política reativa, com pequeno aumento de custo. Seus dados, porém, são privados, e a decisão é aplicada apenas ao próprio previsor.

A principal vantagem do desacoplamento é a modularidade: com a regra de decisão fixa, é possível trocar apenas o previsor e atribuir a ele as diferenças de custo e de violações, propriedade que este trabalho explora.

2.3.3 Aprendizado por reforço

No aprendizado por reforço, um agente aprende uma política que mapeia o estado do sistema diretamente em ações, como adicionar ou remover instâncias, guiado por uma recompensa que penaliza custo e violações (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014). Esses métodos são sensíveis ao projeto da recompensa e aos hiperparâmetros, enfrentam limitações de escalabilidade conforme o espaço de estados cresce (BODRA; KHAIRNAR, 2025) e exigem um ambiente de simulação e um grande número de episódios de treinamento. Além

disso, uma política aprendida é difícil de interpretar: quando ela decide mal, não se sabe se o problema está na antecipação da demanda ou na decisão. Por isso, este trabalho adota a abordagem desacoplada e deixa a comparação com uma política aprendida para trabalhos futuros.

2.4 Comparativo dos trabalhos relacionados e lacuna

Tabela 2.1: Comparativo entre os trabalhos relacionados e a proposta deste trabalho. O traço indica que o critério não se aplica.

Trabalho	Técnica de previsão	Dados	Métricas	Acopla à alocação	Mesma política
Calheiros et al. (2015)	ARIMA	Wikipédia (1 h)	Precisão, QoS e horas de máquina	Sim	Não
Xu et al. (2022)	GRU com convolução, multivariada	Alibaba e Google	Erro e máquinas ativas	Parcial	Não
Pan et al. (2023)	Atenção e processo gaussiano	Alibaba Cloud (privados)	Erro, custo e perda de QoS	Sim	Não
Rossi et al. (2025)	Bayesianas (HBNN, LSTMD)	Alibaba e Google	Erro e nível de serviço	Não	–
Agulló López et al. (2025)	Vários, com saída ajustada	Alibaba e Google	Erro, picos e super-provisionamento	Não	–
Bodra e Khairnar (2025)	Revisão de vários métodos	Variados	Heterogêneas	–	–
Este trabalho	GRU e cinco métodos tradicionais	Alibaba (PF I); Google (PF II)	Erro, custo e violações	Sim	Sim

A Tabela 2.1 resume os trabalhos mais próximos desta proposta, indicando também se a previsão é acoplada a uma decisão de alocação e se diferentes previsores são comparados sob a mesma política. Esses trabalhos formam duas linhas que não se encontram. Os de previsão (XU et al., 2022; ROSSI et al., 2025; AGULLÓ LÓPEZ et al., 2025) comparam vários modelos por métricas calculadas sobre a própria previsão, que não indicam a utilidade dos modelos para a gestão de recursos (ROSSI et al., 2025) nem revelam as falhas nos picos (AGULLÓ LÓPEZ et al., 2025). Os que acoplam a previsão à alocação (CALHEIROS et al., 2015; PAN et al., 2023) aplicam a decisão a um único previsor, comparado com políticas reativas, e o esDNN compara sua simulação de escalonamento apenas com uma política baseada na média das observações anteriores (XU et al., 2022). A revisão de Bodra e Khairnar (2025) observa, por fim, que a literatura carece de comparações quantitativas sistemáticas.

A lacuna que este trabalho busca preencher é, portanto, a de uma avaliação de ponta a ponta, com dados públicos de um *cluster* real, que submeta previsores de complexidade crescente, do método ingênuo à rede neural, à mesma política de alocação, calibrada para o mesmo limite de violações ε, e meça o custo e as

violações resultantes. A pergunta deixa de ser qual modelo erra menos e passa a ser quanto custo cada modelo economiza para um mesmo nível de serviço.

2.5 Tecnologias utilizadas

O protótipo é escrito em Python 3.12, pela disponibilidade de bibliotecas para séries temporais e aprendizado profundo, e o código está em repositório público (ROSAS, 2026). A Tabela 2.2 relaciona as ferramentas usadas em cada etapa do *pipeline*.

Tabela 2.2: Ferramentas utilizadas em cada etapa do *pipeline*.

Etapa	**Ferramenta**	**Uso no trabalho**
Dados	DuckDB (RAASVELDT; MÜHLEISEN, 2019)	Agregação do rastro bruto (1,7 GB compactado) por consultas SQL, sem carregá-lo inteiro na memória
	Parquet, pandas (MCKINNEY, 2010) e NumPy (HARRIS et al., 2020)	Armazenamento e manipulação das séries; previsores ingênuo, média móvel e sazonal ingênuo
Previsão	statsmodels (SEABOLD; PERKTOLD, 2010); PyTorch (PASZKE et al., 2019) para CPU	ARIMA e Holt-Winters; GRU, sem exigir GPU
Simulação	Python (PF I); CloudSim Plus (FILHO et al., 2017) (PF II)	Alocação intervalo a intervalo e métricas
Resultados	Matplotlib (HUNTER, 2007)	Figuras
Ambiente	uv, Docker, Makefile, pytest, Git e GitHub	Dependências travadas, contêiner, encadeamento das etapas, testes e versionamento

O CloudSim (CALHEIROS et al., 2011) é um simulador de eventos discretos que modela centros de dados, servidores, máquinas virtuais e a execução de tarefas, usado, por exemplo, na avaliação do provisionamento guiado por ARIMA de Calheiros et al. (2015); o CloudSim Plus é um *framework* em Java derivado dele, cujo uso é descrito nas Seções 3.1 e 4.3. Para a reprodutibilidade, as versões das dependências ficam travadas, todos os parâmetros dos experimentos ficam em um arquivo de configuração, e nenhum no código, e um único comando regenera todas as tabelas e figuras a partir dos dados brutos.

3
Objetivos do trabalho

Este capítulo descreve a solução proposta para a lacuna identificada na Seção 2.4, os objetivos que a orientam, o escopo e o público do sistema, e os itens que serão apresentados à banca ao final do Projeto Final II.

3.1
Solução proposta

A solução proposta é um arcabouço de alocação preditiva de recursos em nuvem construído para responder a uma pergunta: quanto custo operacional uma previsão de carga melhor permite economizar, para um mesmo nível de serviço. O arcabouço segue a abordagem desacoplada da Seção 2.3.2, em que previsão e decisão de alocação são etapas separadas, e é organizado nos quatro componentes da Figura 3.1.

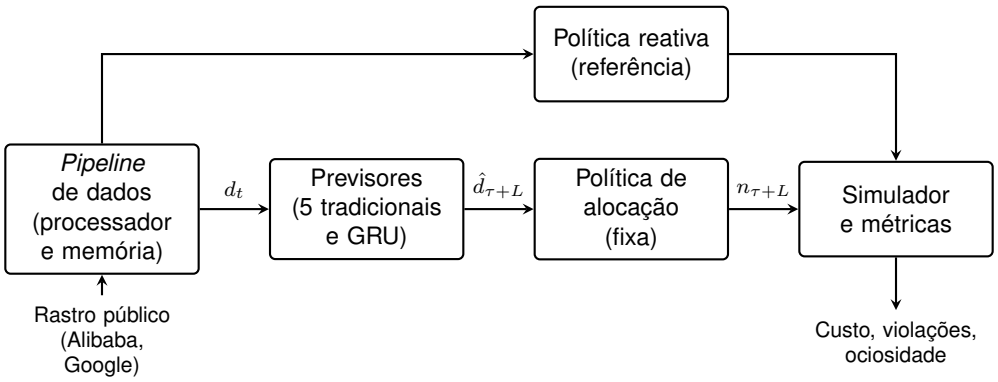


Figura 3.1: Arquitetura da solução proposta.

***Pipeline* de dados.** Converte um rastro público de *cluster* em séries de demanda de processador e de memória com intervalos de cinco minutos, expressas em máquinas equivalentes totalmente utilizadas.

Previsores. Cinco métodos tradicionais (ingênuo, média móvel, sazonal ingênuo, Holt-Winters e ARIMA; Seção 2.2.1) e uma GRU que, a partir das últimas seis horas, prevê todos os horizontes de uma só vez (estratégia direta; Seção 2.2.2). No Projeto Final I, a GRU é univariada e usa apenas a demanda de processador (Seção 3.3). Todos os previsores compartilham uma interface que só dá acesso ao histórico até a origem τ da previsão.

Política de alocação. Aplica uma margem de segurança m sobre a demanda prevista com horizonte $h = L$ (Equação 2-4). Como referências, há uma política reativa no estilo do HPA do Kubernetes (Equação 2-3) (The Kubernetes Authors, 2024), o dimensionamento estático, que mantém fixo o número de instâncias necessário no pico do treino, e o oráculo da Seção 2.1.4, que conhece a demanda futura.

O parâmetro de cada política é escolhido no conjunto de validação e avaliado no conjunto de teste. Para um limite de violações ε, a margem escolhida é a de menor custo, na grade da Tabela 3.1, entre as que cumprem o limite na validação:

$$m^*(\varepsilon) = \arg\min_{m} \; C_{\mathrm{val}}(m) \quad \text{sujeito a} \quad V_{\mathrm{val}}(m) \leq \varepsilon, \tag{3-1}$$

em que C_{val} e V_{val} são o custo e a fração de intervalos com violação na validação. O **ponto de operação** de um previsor para o limite ε é a política preditiva com a margem $m^*(\varepsilon)$, medida no teste. Na política reativa, a escolha é a mesma, com a utilização-alvo u^* no lugar de m; o dimensionamento estático e o oráculo são usados sem margem. Se nenhum valor da grade cumpre o limite na validação, a política não tem ponto de operação para aquele ε. Essa é a versão operacional da Equação 1-1: como a restrição é verificada na validação, o teste nunca participa da escolha, e o ponto de operação pode ultrapassar ε no teste.

Simulador e métricas. Reproduz, intervalo a intervalo, um centro de dados com instâncias homogêneas, em que a decisão tomada na origem τ só vale no intervalo $\tau + L$, e contabiliza as métricas da Seção 2.1.4. No Projeto Final I, esse papel é cumprido por um protótipo em Python; no Projeto Final II, pelo CloudSim Plus (FILHO et al., 2017), que recebe por arquivo as previsões geradas em Python, sem ponte entre as linguagens em tempo de execução (Seção 2.5).

A decisão metodológica central é manter a política de alocação **fixa** e variar apenas o previsor. Assim, qualquer diferença de custo ou de violações entre duas configurações pode ser atribuída à previsão, e não à regra de decisão.

3.2 Objetivos

3.2.1 Objetivo geral

Avaliar, em um centro de dados simulado alimentado por rastros públicos de *clusters* reais, a redução de custo operacional obtida pela alocação preditiva guiada por uma rede neural recorrente, em comparação com métodos tradicionais de previsão submetidos à mesma política e com uma política reativa, para limites fixos de violação de SLA.

3.2.2 Questões de pesquisa

O objetivo geral se desdobra em quatro questões de pesquisa (QP):

QP1 Para um mesmo limite de violações ε, a alocação guiada pela rede neural tem custo menor do que a guiada pelos métodos tradicionais? De quanto é a diferença?

QP2 Um previsor com menor erro (RMSE, MAE) leva necessariamente a uma alocação mais barata no ponto de operação?

QP3 Qual é o ganho da alocação preditiva em relação à reativa, e como ele varia com a antecedência da decisão L?

QP4 As conclusões obtidas no rastro da Alibaba se mantêm no rastro do Google?

3.2.3 Objetivos específicos

Para responder a essas questões, o trabalho tem os seguintes objetivos específicos (OE):

OE1 **Dados:** agregar o Alibaba Cluster Trace de 2018 (Alibaba Group, 2018) em séries de cinco minutos, com tratamento de lacunas, divisão cronológica e relatório de qualidade.

OE2 **Previsores:** implementar os seis previsores e avaliá-los por MAE, RMSE, MAPE, viés e taxa de subestimação em cada horizonte.

OE3 **Simulador e políticas:** implementar o protótipo do simulador e as políticas da Seção 3.1 (Equação 2-4), validados por testes automatizados.

OE4 **Simulação no CloudSim Plus:** reproduzir no CloudSim Plus (FILHO et al., 2017) as políticas e as métricas do protótipo, recebendo por arquivo as previsões geradas em Python, e usá-lo na avaliação final.

OE5 **Custo e violações (QP1, QP3):** obter os pontos de operação para $\varepsilon = 1\%$ e 5% (Equação 3-1) e as curvas de custo contra violações.

OE6 **Erro e custo (QP2):** confrontar o erro da previsão de horizonte $h = L$ com o custo no ponto de operação.

OE7 **Sensibilidade (QP3):** medir o efeito da antecedência da decisão L sobre os pontos de operação.

OE8 **Robustez:** repetir o treinamento da GRU com várias sementes e avaliar em várias janelas de teste, reportando média e dispersão.

OE9 **Replicação (QP4):** aplicar o mesmo protocolo ao Google Cluster Trace de 2019 (WILKES, 2020), alterando apenas a leitura dos dados.

OE10 **Reprodutibilidade:** manter código e configurações em repositório público (ROSAS, 2026), regenerando todas as tabelas e figuras com um único comando.

3.3 Escopo do sistema

O sistema é um **ambiente de simulação e avaliação**, e não um autoescalador para produção. Recebe um rastro de utilização e um arquivo de configuração e produz tabelas e figuras com as métricas de previsão e de alocação; nenhum parâmetro é fixado no código. A Tabela 3.1 resume os parâmetros do protótipo construído no Projeto Final I, que serão mantidos no Projeto Final II, salvo os ajustes decorrentes de OE7 a OE9.

Tabela 3.1: Parâmetros do protocolo experimental adotados no protótipo.

Parâmetro	Símbolo	Valor
Intervalo de agregação	Δ	5 minutos
Divisão cronológica (treino/validação/teste)	–	60% / 20% / 20%
Horizontes de previsão avaliados	h	1, 2, 3 e 6 intervalos (5 a 30 min)
Janela de entrada da GRU	w	72 intervalos (6 h)
Período sazonal (sazonal ingênuo, Holt-Winters)	s	288 intervalos (1 dia)
Antecedência da decisão	L	1 intervalo (sem atraso de inicialização); 2 na análise de sensibilidade
Instâncias no pico do treino	n_{pico}	50
Capacidade de uma instância	k	pico da demanda no treino dividido por n_{pico} (60,94 máquinas equivalentes)
Custo por instância e intervalo	c	1
Número mínimo de instâncias	$n_{\min}$	1
Grade de margens (política preditiva)	m	0 a 0,60, com passo 0,01
Grade de utilizações-alvo (política reativa)	u^*	0,50 a 0,99, com passo 0,01
Tolerância e estabilização (política reativa)	–	10% e 6 intervalos
Limites de violação	ε	1% e 5%

Valem as simplificações do Capítulo 2: demanda agregada, capacidade em instâncias homogêneas e violação de SLA sempre que a demanda de um intervalo excede a capacidade. No protótipo do Projeto Final I, por simplicidade de escopo, a alocação considera apenas a demanda de processador, e a GRU é univariada. Essa simplificação será retirada no Projeto Final II, que também considera a memória; a forma de incorporá-la, como segundo recurso na alocação, como entrada adicional da previsão ou de ambas as formas, ainda está em definição. Os itens fora do escopo são os da Seção 1.4.

3.4 Usuários e situações apoiadas

O ambiente se destina a pesquisadores e a engenheiros responsáveis pelo planejamento de capacidade que precisam decidir se a adoção de um previsor mais complexo se paga em custo de infraestrutura. Ele apoia três situações: comparar previsores sob um mesmo limite de violações antes de levá-los à produção; estimar o custo de operar com um dado nível de serviço e atraso de

inicialização; e estender a avaliação a novos previsores, políticas ou rastros, o que exige apenas implementar a interface comum do componente correspondente.

3.5 Avanços em relação às soluções existentes

Em relação à lacuna da Seção 2.4, o trabalho busca:

- **Comparação controlada:** previsores do ingênuo à rede neural sob a mesma política, simulador e dados.
- **Custo a um mesmo nível de serviço:** com instâncias discretas e antecedência da decisão.
- **Ligação entre erro e custo:** a QP2 testa se menor erro implica alocação mais barata.

O trabalho não propõe uma nova arquitetura: a GRU é deliberadamente simples, e a contribuição está em medir o valor operacional da previsão com rastros públicos e código aberto.

3.6 Itens a serem implementados e apresentados à banca

A Tabela 3.2 relaciona os itens a serem entregues e sua situação ao fim do Projeto Final I. O protótipo já cobre o *pipeline* completo, dos dados brutos às curvas de custo contra violações; o que foi realizado, as dificuldades encontradas e os resultados preliminares são discutidos no Capítulo 4.

Tabela 3.2: Itens a serem implementados e situação ao fim do Projeto Final I.

Item	Objetivo	Situação no PF I
Pipeline de dados	OE1	Implementado
Previsores	OE2	Implementado
Simulador e políticas	OE3	Protótipo em Python implementado
Simulação no CloudSim Plus	OE4	Previsto para o PF II
Experimentos de custo, violações e erro contra custo	OE5, OE6	Preliminar (uma semente, uma janela de teste)
Análise de sensibilidade	OE7	Parcial ($L = 1$ e $L = 2$)
Avaliação robusta	OE8	Previsto para o PF II
Replicação no Google	OE9	Previsto para o PF II
Repositório reprodutível	OE10	Implementado; a ser mantido
Relatório final e apresentação	–	Previsto para o PF II

Na defesa, serão apresentados: (i) as curvas de custo contra violações e os pontos de operação de todos os previsores e da política reativa, nos dois rastros; (ii) a comparação entre a ordenação dos previsores por erro e por custo; (iii) a análise de sensibilidade à antecedência da decisão L; e (iv) a regeneração dos resultados a partir dos dados brutos pelo repositório.

Uma extensão, condicionada ao cronograma e fora do compromisso de entrega, pode ser avaliada no mesmo arcabouço: a substituição da margem fixa

por um quantil previsto pela rede (ROSSI et al., 2025; PAN et al., 2023). A incorporação da memória, por outro lado, deixa de ser uma extensão e faz parte do Projeto Final II (Seção 3.3); a previsão multivariada nos moldes do esDNN (XU et al., 2022) é uma das formas possíveis.

4
Plano de ação

Este capítulo revisa o plano de ação do Projeto Final I. A Seção 4.1 resume o que estava previsto, e a Seção 4.2 compara as atividades previstas com as realizadas. A Seção 4.3 discute as mudanças de plano, a Seção 4.4 relata as dificuldades encontradas e como foram contornadas, e a Seção 4.5 apresenta os resultados preliminares do protótipo.

4.1
Plano previsto

A proposta do trabalho previa avaliar a alocação preditiva com rastros públicos de dois *clusters*, o da Alibaba (Alibaba Group, 2018) e o do Google (WILKES, 2020), em um centro de dados simulado no CloudSim Plus (FILHO et al., 2017), comparando uma rede neural com métodos tradicionais de previsão e com uma política reativa (Seção 1.4).

Para o Projeto Final I, a meta foi um protótipo de ponta a ponta que servisse de evidência de viabilidade: partir do rastro bruto da Alibaba e chegar às curvas de custo contra violações, primeiro com previsores tradicionais e, na etapa seguinte, com a rede neural. A construção foi dividida em fases curtas, cada uma com uma especificação, um critério de aceite e um prazo máximo, e com pontos de verificação ao fim das fases de ambiente, dados, simulador e experimento, em que os resultados eram conferidos antes de seguir. A Tabela 4.1 resume as fases, com o prazo reservado para cada uma e a data em que foi concluída. A entrega do protótipo estava prevista para 21/10/2026.

Tabela 4.1: Fases previstas para a construção do protótipo no Projeto Final I e datas de conclusão, em 2026.

Fase	**Entrega prevista**	**Conclusão**
Ambiente	Projeto com dependências travadas, contêiner e testes de fumaça	14/10
Dados	Série de demanda de cinco minutos e relatório de qualidade	15/10, com revisão em 21/10
Previsores tradicionais	Cinco previsores, avaliação com origem móvel e tabela de erros	15/10
Simulador	Simulador, políticas, métricas, testes e verificações de sanidade	15/10
Experimento	Curvas de custo contra violações, pontos de operação e relato ao orientador	16/10, com revisão em 20/10
Rede neural	Previsor GRU com a mesma interface dos demais	20/10

4.2 Atividades realizadas

Todas as fases previstas foram concluídas dentro do prazo, e a rede neural, que estava prevista para a etapa seguinte, foi antecipada. As datas da Tabela 4.1 vêm do histórico de versões do repositório do projeto (ROSAS, 2026). Em relação aos objetivos específicos da Seção 3.2.3, foi realizado o seguinte.

Dados (OE1). A tabela de uso por máquina do rastro da Alibaba, com 246.934.820 linhas, foi agregada em uma série de 2.304 intervalos de cinco minutos, que cobre os oito dias do rastro e as $M = 4.023$ máquinas (Figura 4.1). A fração das máquinas que enviam medições, chamada aqui de cobertura, varia ao longo do rastro: na validação e no teste, ela é de pelo menos 97,8% em todos os intervalos; no treino, que contém o início do rastro, a média é de 88,6% e a mínima, de 53,7%. Um apagão de coleta deixou no treino uma lacuna de 65 intervalos, cerca de 5,4 h, preenchida como descrito na Seção 4.3. A divisão cronológica resultou em 1.382 intervalos de treino, 460 de validação e 462 de teste.

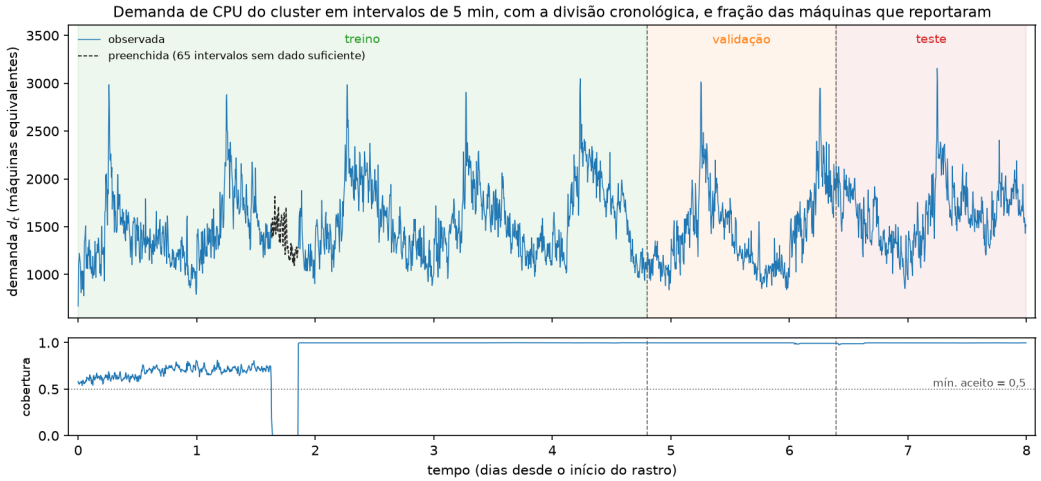


Figura 4.1: Demanda de processador do *cluster* nos oito dias do rastro, em intervalos de cinco minutos, com a divisão cronológica. O trecho tracejado é a lacuna de coleta preenchida. O painel inferior mostra a cobertura, isto é, a fração das M máquinas que enviaram medições em cada intervalo; abaixo da linha pontilhada, o intervalo é tratado como lacuna.

Previsores (OE2). Os cinco métodos tradicionais e a GRU foram implementados com a mesma interface e avaliados com origem móvel (*walk-forward*): cada previsão usa apenas os dados até a sua origem, e a origem avança um intervalo por vez sobre a validação e o teste, com 460 e 462 origens, respectivamente, para os horizontes de 1, 2, 3 e 6 intervalos. Testes automatizados verificam, para cada previsor, que alterar os dados posteriores à origem não muda nenhuma previsão.

Simulador e políticas (OE3). O protótipo do simulador implementa a antecedência L, as quatro políticas (preditiva, reativa, dimensionamento estático e oráculo) e as métricas da Seção 2.1.4, com testes automatizados de casos calcu-

lados à mão e verificações de sanidade. Uma delas confirma que o oráculo não viola o SLA em nenhum intervalo, como esperado.

Experimentos (OE5 a OE7). A varredura das grades de m e de u^* gerou as curvas de custo contra violações e os pontos de operação para $\varepsilon = 1\%$ e 5%, com $L = 1$, e a simulação foi repetida com $L = 2$ para uma primeira análise de sensibilidade. Os resultados estão na Seção 4.5.

Reprodutibilidade (OE10). Todo o *pipeline*, do arquivo bruto às tabelas e figuras, roda com um único comando, e duas execuções completas seguidas produzem arquivos idênticos byte a byte.

4.3 Mudanças em relação ao plano

Simulador. A proposta previa o CloudSim Plus desde o início. No Projeto Final I, o protótipo usou um simulador próprio, em Python, por três motivos. Primeiro, a decisão estudada se resume a escolher n_t a cada intervalo, a partir da demanda agregada, o que um simulador pequeno, em tempo discreto, representa diretamente, sem modelar a execução de tarefas nem o posicionamento de máquinas virtuais. Segundo, manter previsão e simulação na mesma linguagem permitiu iterar rapidamente sobre o protocolo experimental. Terceiro, o simulador pôde ser coberto por testes automatizados e conferido intervalo a intervalo, o que ajudou a garantir que as diferenças entre previsores não fossem artefatos da simulação. A escolha é temporária: no Projeto Final II, o CloudSim Plus substitui o protótipo na avaliação final (OE4).

Rede neural antecipada. O plano deixava a rede neural para depois do protótipo com previsores tradicionais. Como a interface comum de previsores já estava pronta, a GRU entrou como mais um previsor ainda no Projeto Final I, sem nenhuma alteração na política ou no simulador. Por simplicidade, ela é univariada e usa apenas a demanda de processador; a memória fica para o Projeto Final II (Seção 3.3).

Rastro completo. A primeira versão do protótipo descartava o início do rastro, que tem um regime de coleta diferente, com cobertura de máquinas bem menor, e um apagão de coleta dentro do conjunto de treino: dos 2.304 intervalos, restavam 1.769, uma perda de 23% da série, e o teste ficava com 355 intervalos. A versão final usa o rastro inteiro. Os intervalos em que menos da metade das máquinas enviou medições são tratados como lacuna. As lacunas de até três intervalos são interpoladas linearmente, e a única lacuna maior, a do apagão, é preenchida com a demanda do mesmo horário do dia anterior, somada a uma correção que varia linearmente entre as diferenças observadas nas duas bordas; assim, o trecho preenchido mantém a forma do ciclo diário e coincide com a demanda observada nas bordas. Como esse preenchimento usa a borda posterior à lacuna, o protótipo só o aceita quando todos os dados usados estão no treino, o que garante que nenhuma previsão da validação ou do teste use dados posteriores à sua origem. Os intervalos preenchidos entram no ajuste dos previsores, mas não

nas estatísticas descritivas nem nas métricas. O ganho é um teste 30% maior, com 462 intervalos; o custo é que, no início do treino, a demanda é extrapolada a partir de uma fração menor das máquinas (Figura 4.1).

Grade de parâmetros. A grade inicial de margens tinha passos de cinco a 20 pontos percentuais, que geravam degraus de custo maiores que as diferenças entre previsores, de modo que o ponto de operação era decidido pelo degrau da grade, e não pelo previsor. Com os dados atuais, essa grade levaria o ARIMA e o método ingênuo à mesma margem em $\varepsilon = 1\%$, com custos praticamente iguais, e faria o ARIMA parecer 7,5% mais barato em $\varepsilon = 5\%$. Com a grade fina adotada, de passo 0,01 em m e em u^* (Tabela 3.1), as diferenças são de 4,3% e 2,4%, respectivamente.

Escopo adiado. A replicação no rastro do Google (OE9), a avaliação com várias sementes e janelas de teste (OE8) e a incorporação da memória ficaram para o Projeto Final II.

4.4 Dificuldades encontradas

Holt-Winters. O modelo ajustado pela biblioteca statsmodels não permite aplicar os parâmetros estimados a dados novos, e reajustá-lo em cada uma das centenas de origens da avaliação seria lento demais. A solução foi estimar os parâmetros no treino com a biblioteca e implementar a recursão do método no próprio previsor. Um teste automatizado confirma que ela reproduz os valores ajustados pela biblioteca até a precisão de máquina.

Ponto flutuante. Três problemas tiveram a mesma origem. Primeiro, o oráculo aparecia com violações inexistentes, porque $k \cdot \lceil d_t/k \rceil$ pode ficar ligeiramente abaixo de d_t por erro de arredondamento; a comparação entre demanda e capacidade passou a usar uma tolerância relativa de 10^{-9}. Segundo, o arredondamento para cima do número de instâncias saltava para o inteiro seguinte quando o valor exato era inteiro, o que dava uma instância a mais em parte das configurações do dimensionamento estático; o arredondamento passou a usar a mesma tolerância. Terceiro, a soma paralela feita pelo DuckDB na agregação variava na última casa decimal entre execuções e propagava a diferença aos resultados; a série passou a ser gravada com seis casas decimais, e as execuções passaram a gerar arquivos idênticos.

Memória na agregação. A agregação do rastro chega a ocupar cerca de 14 GB de memória e não terminou em contêineres limitados a 4 ou 8 GB. Nesses ambientes, a etapa de dados roda fora do contêiner, e as demais etapas continuam dentro dele.

Expectativas que não se confirmaram. Duas suposições do plano foram desmentidas pelos dados. A primeira era que o oráculo teria o menor custo possível: a política preditiva com o sazonal ingênuo e sem margem tem $C_{\mathrm{norm}} = 0{,}900$, mas viola em 64,9% dos intervalos do teste. O oráculo é, portanto, o piso do custo sem violações, como definido na Seção 2.1.4. A segunda era que a política

reativa sempre violaria por causa da antecedência: com $u^* \leq 0{,}7$, ela não viola nenhum intervalo do teste, porque a folga de $1/u^* - 1$, de pelo menos 43%, cobre a variação da demanda em um intervalo.

Tamanho do teste. Com 462 intervalos, um intervalo com violação vale 0,22 ponto percentual de V, e a diferença entre cumprir ou não $\varepsilon = 1\%$ no teste é de um único intervalo. O uso do rastro completo aumentou o teste, mas não mudou a ordem de grandeza do problema. Essa limitação motiva a avaliação com várias janelas de teste prevista para o Projeto Final II (OE8).

4.5 Resultados preliminares

Os resultados desta seção vêm do protótipo, com o rastro da Alibaba, $L = 1$ e os parâmetros da Tabela 3.1. Os parâmetros das políticas foram escolhidos na validação (Equação 3-1) e medidos no teste, que não participa de nenhuma escolha. Os resultados são preliminares porque usam uma única janela de teste e uma única semente de treinamento da GRU.

A Tabela 4.2 mostra o erro das previsões no teste para o horizonte usado na alocação, $h = L = 1$. O ARIMA tem o menor erro, seguido da GRU, com RMSE 3,1% maior. Os vieses do ARIMA, do Holt-Winters, do método ingênuo e da média móvel são pequenos perto do RMSE, entre $-1{,}13$ e $+3{,}01$. A GRU superestima a demanda em média, com viés de $+17{,}59$, e fica abaixo do valor real em apenas 39,4% das origens. Na alocação, a subestimação vira violação e precisa ser compensada pela margem, e a superestimação vira ociosidade. O sazonal ingênuo é o pior por larga margem: ele repete o valor de um dia antes e não acompanha as variações de curto prazo nem as mudanças de nível entre os dias, e subestima a demanda em 66,9% das origens do teste.

Tabela 4.2: Erro das previsões no conjunto de teste com horizonte $h = 1$, em 462 origens. RMSE, MAE e viés estão em máquinas equivalentes; viés positivo indica superestimação.

Previsor	RMSE	MAE	Viés	Taxa de subestimação
ARIMA	157,44	120,25	$+1{,}75$	42,9%
GRU	162,26	125,56	$+17{,}59$	39,4%
Ingênuo	168,05	126,83	$+0{,}56$	45,0%
Holt-Winters	173,68	133,44	$-1{,}13$	51,5%
Média móvel	187,65	142,68	$+3{,}01$	46,5%
Sazonal ingênuo	360,90	291,42	$-166{,}41$	66,9%

A Figura 4.2 mostra as curvas completas de custo contra violações no teste, e a Tabela 4.3, os pontos de operação de cada política. Cinco observações se destacam.

A previsão reduz muito o custo em relação ao dimensionamento estático e pouco em relação à política reativa. Em $\varepsilon = 5\%$, a política preditiva com o ARIMA tem $C_{\mathrm{norm}} = 1{,}176$, contra 1,830 do dimensionamento estático, uma redu-

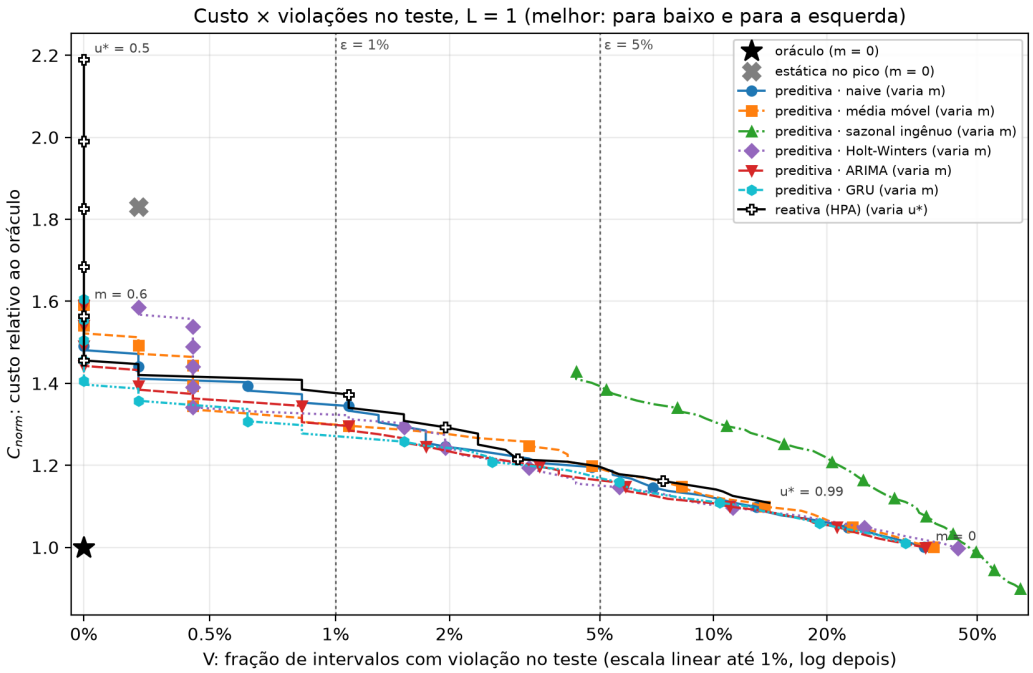


Figura 4.2: Custo normalizado pelo do oráculo contra a fração de intervalos com violação no conjunto de teste, com $L = 1$. Cada curva varia o parâmetro de uma política (m na preditiva e u^* na reativa), e as linhas verticais marcam $\varepsilon = 1\%$ e 5%. O eixo horizontal é linear até 1% e logarítmico a partir daí. Na legenda, *naive* designa o método ingênuo.

Tabela 4.3: Pontos de operação no conjunto de teste, com $L = 1$. O parâmetro de cada política foi escolhido na validação (Equação 3-1). Valores de V em negrito ultrapassam ε no teste.

	$\varepsilon = 1\%$			$\varepsilon = 5\%$		
Política	**Parâmetro**	C_{norm}	V	**Parâmetro**	C_{norm}	V
Oráculo	–	1,000	0,00%	–	1,000	0,00%
Dimensionamento estático	–	1,830	0,22%	–	1,830	0,22%
Reativa (HPA)	$u^* = 0{,}82$	1,340	**1,08%**	$u^* = 0{,}92$	1,197	4,98%
Preditiva com ARIMA	$m = 0{,}32$	1,315	0,87%	$m = 0{,}18$	1,176	3,90%
Preditiva com GRU	$m = 0{,}28$	1,287	0,87%	$m = 0{,}19$	1,199	3,25%
Preditiva com Holt-Winters	$m = 0{,}40$	1,391	0,43%	$m = 0{,}19$	1,185	3,46%
Preditiva com ingênuo	$m = 0{,}38$	1,373	0,87%	$m = 0{,}21$	1,206	3,46%
Preditiva com média móvel	$m = 0{,}36$	1,354	0,43%	$m = 0{,}24$	1,238	3,90%
Preditiva com sazonal ingênuo	$m = 0{,}42$	1,270	**13,64%**	$m = 0{,}23$	1,103	**33,55%**

ção de 35,7%, e 1,197 da reativa, uma redução de 1,7%. Em $\varepsilon = 1\%$, a GRU tem $C_{\text{norm}} = 1{,}287$, 3,9% abaixo da reativa, que ainda ultrapassa o limite no teste.

A GRU é competitiva com o ARIMA, sem vantagem clara. A diferença de custo entre os dois fica em torno de 2% nos dois limites, em sentidos opostos: a GRU é mais barata em $\varepsilon = 1\%$, com 1,287 contra 1,315 e o mesmo número de intervalos com violação no teste, e o ARIMA é mais barato em $\varepsilon = 5\%$, com 1,176 contra 1,199. Como a escolha do parâmetro e a medida no teste dependem de poucos intervalos com violação (Seção 4.4), diferenças desse tamanho não permitem afirmar que um previsor é melhor que o outro.

Menor erro de previsão não garante menor custo. O Holt-Winters tem o quarto menor RMSE (Tabela 4.2), mas, em $\varepsilon = 1\%$, é o previsor mais caro entre os que cumprem o limite no teste, acima da média móvel, cujo RMSE é 8% maior. Em $\varepsilon = 5\%$, ao contrário, ele é o segundo mais barato, à frente da GRU e do método ingênuo, que têm RMSE menor. Esse resultado antecipa a resposta à QP2 e sugere que o que pesa na alocação é o erro nas subidas da demanda, e não o erro médio.

A calibração na validação falha quando a demanda muda de comportamento entre os conjuntos. O sazonal ingênuo, que tem o pior erro, aparece com o menor custo nos dois limites, mas só porque viola em 13,6% e 33,5% dos intervalos do teste, muito acima de ε. Para esse previsor, o viés com $h = 1$ é a diferença média entre a demanda de um dia antes e a atual: ele passa de $+96{,}08$ na validação, em que a demanda cai em relação ao dia anterior, para $-166{,}41$ no teste, em que ela sobe, e a margem escolhida na validação deixa de cobrir a subestimação. Os previsores que acompanham a demanda recente cumprem os dois limites no teste, mas o caso mostra que calibrar a margem em uma única janela supõe que o erro tem a mesma distribuição na validação e no teste.

A política reativa muda de tamanho com muito menos frequência. Nos pontos de operação, ela altera o número de instâncias 116 vezes no teste, contra 326 a 411 vezes das políticas preditivas, com exceção da média móvel, com 282 a 284. Como o modelo de custo não cobra pelas alterações, essa vantagem da reativa não aparece em C.

Com $L = 2$, as violações aumentam, em média sobre a grade de parâmetros, em todas as políticas que dependem da demanda recente. O aumento é maior nas políticas preditivas com o método ingênuo (8,0 intervalos com violação a mais no teste), o Holt-Winters (7,8), o ARIMA (5,4) e a GRU (5,1) do que na reativa (4,2); a média móvel fica abaixo da reativa (3,8), e o sazonal ingênuo não muda, porque sua previsão não depende do horizonte. Por fim, três dos 16 pontos de operação que não são do oráculo ultrapassam ε no teste: o da reativa em $\varepsilon = 1\%$, por um intervalo, e os dois do sazonal ingênuo, por larga margem, o que reforça a necessidade da avaliação com várias janelas de teste no Projeto Final II.

5
Cronogramas

Este capítulo apresenta o cronograma do Projeto Final I, com as atividades realizadas, e o cronograma proposto para o Projeto Final II. Nas duas tabelas, as células preenchidas marcam os meses em que cada atividade acontece, e a coluna **Objetivo** relaciona a atividade aos objetivos específicos da Seção 3.2.3.

5.1
Projeto Final I

O Projeto Final I ocupou o segundo semestre de 2026, de agosto a novembro (Tabela 5.1). Os primeiros meses foram dedicados à escolha do tema, à revisão bibliográfica e à elaboração da proposta. Em outubro, o protótipo foi construído e avaliado, com as datas detalhadas na Tabela 4.1, e a redação deste relatório se estendeu de outubro a novembro.

Tabela 5.1: Cronograma do Projeto Final I, em 2026.

Atividade	Objetivo	Ago	Set	Out	Nov
Escolha do tema e definição da pergunta de pesquisa	–	■			
Revisão bibliográfica	–	■	■	■	
Elaboração da proposta	–		■		
Ambiente de desenvolvimento e tratamento dos dados	OE1			■	
Previsores tradicionais e GRU	OE2			■	
Protótipo do simulador e das políticas	OE3			■	
Experimentos preliminares	OE5 a OE7			■	
Repositório reprodutível	OE10			■	
Redação do relatório do Projeto Final I	–			■	■
Entrega do relatório	–				■

5.2
Projeto Final II

O Projeto Final II vai de dezembro de 2026 a junho de 2027, com a defesa prevista para o fim de junho (Tabela 5.2). O cronograma começa pela definição, com o orientador, da forma de incorporar a memória e pela migração para o CloudSim Plus, porque todos os resultados finais dependem das duas. Em seguida vêm a replicação no rastro do Google e a avaliação com várias sementes e janelas de teste, que alimentam os experimentos finais e a análise de sensibilidade. A substituição da margem por um quantil previsto pela rede só será avaliada se o cronograma permitir (Seção 3.6). A manutenção do repositório reprodutível acompanha todo o período.

Tabela 5.2: Cronograma proposto para o Projeto Final II.

		2026	2027					
Atividade	**Objetivo**	**Dez**	**Jan**	**Fev**	**Mar**	**Abr**	**Mai**	**Jun**
Definição da forma de incorporar a memória	–							
Simulação no CloudSim Plus	OE4							
Incorporação da memória	–							
Replicação no rastro do Google	OE9							
Avaliação com várias sementes e janelas de teste	OE8							
Experimentos finais de custo, violações e erro contra custo	OE5, OE6							
Análise de sensibilidade à antecedência L	OE7							
Extensão opcional: quantil previsto pela rede	–							
Manutenção do repositório reprodutível	OE10							
Redação do relatório final	–							
Preparação da apresentação e defesa	–							

Referências

AGULLÓ LÓPEZ, F. et al. Enhancing the output of time series forecasting algorithms for cloud resource provisioning. **Future Generation Computer Systems**, v. 170, p. 107833, 2025.

Alibaba Group. **Alibaba Cluster Trace Program**. 2018. Disponível em: https://github.com/alibaba/clusterdata.

Amazon Web Services. **Predictive scaling for Amazon EC2 Auto Scaling**. 2024. Documentação do Amazon EC2 Auto Scaling. Disponível em: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html. Acesso em: 14 out. 2026.

BELOGLAZOV, A.; BUYYA, R. Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. **Concurrency and Computation: Practice and Experience**, v. 24, n. 13, p. 1397–1420, 2012.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. **IEEE Transactions on Neural Networks**, v. 5, n. 2, p. 157–166, 1994.

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016.

BODRA, D.; KHAIRNAR, S. Machine learning-based cloud resource allocation algorithms: a comprehensive comparative review. **Frontiers in Computer Science**, v. 7, p. 1678976, 2025.

BOX, G. E. P. et al. **Time Series Analysis: Forecasting and Control**. 5. ed. Hoboken, NJ: John Wiley & Sons, 2015.

CALHEIROS, R. N. et al. Workload prediction using ARIMA model and its impact on cloud applications' QoS. **IEEE Transactions on Cloud Computing**, v. 3, n. 4, p. 449–458, 2015.

CALHEIROS, R. N. et al. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. **Software: Practice and Experience**, v. 41, n. 1, p. 23–50, 2011.

CHO, K. et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: **Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)**. [S.l.: s.n.], 2014. p. 1724–1734.

CHUNG, J. et al. Empirical evaluation of gated recurrent neural networks on sequence modeling. In: **NIPS 2014 Workshop on Deep Learning**. [S.l.: s.n.], 2014. Disponível em: https://arxiv.org/abs/1412.3555.

FILHO, M. C. S. et al. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: **2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)**. [S.l.: s.n.], 2017. p. 400–406.

HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.

HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

HYNDMAN, R. J.; ATHANASOPOULOS, G. **Forecasting: principles and practice**. 2. ed. Melbourne: OTexts, 2018. Disponível em: https://otexts.com/fpp2/.

LORIDO-BOTRAN, T.; MIGUEL-ALONSO, J.; LOZANO, J. A. A review of auto-scaling techniques for elastic applications in cloud environments. **Journal of Grid Computing**, v. 12, n. 4, p. 559–592, 2014.

MAO, M.; HUMPHREY, M. A performance study on the VM startup time in the cloud. In: **2012 IEEE Fifth International Conference on Cloud Computing (CLOUD)**. [S.l.: s.n.], 2012. p. 423–430.

MCKINNEY, W. Data structures for statistical computing in Python. In: WALT, S. van der; MILLMAN, J. (Ed.). **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010. p. 56–61.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing**. Gaithersburg, MD, 2011.

PAN, Z. et al. MagicScaler: Uncertainty-aware, predictive autoscaling. **Proceedings of the VLDB Endowment**, v. 16, n. 12, p. 3808–3821, 2023.

PASZKE, A. et al. PyTorch: An imperative style, high-performance deep learning library. In: **Advances in Neural Information Processing Systems 32 (NeurIPS 2019)**. [S.l.: s.n.], 2019.

RAASVELDT, M.; MÜHLEISEN, H. DuckDB: an embeddable analytical database. In: **Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)**. [S.l.]: ACM, 2019. p. 1981–1984.

ROSAS, G. L. d. S. **predictive-cloud-autoscaling: código do Projeto Final**. 2026. Disponível em: https://github.com/gabriellsar/predictive-cloud-autoscaling. Acesso em: 20 out. 2026.

ROSSI, A. et al. Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning. **Cluster Computing**, v. 28, n. 4, p. 258, 2025. Código e dados disponíveis em: https://github.com/andreareds/TowardsUncertaintyAwareWorkloadPrediction.

SEABOLD, S.; PERKTOLD, J. statsmodels: Econometric and statistical modeling with Python. In: **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010.

TAIEB, S. B. et al. A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. **Expert Systems with Applications**, v. 39, n. 8, p. 7067–7083, 2012.

The Kubernetes Authors. **Horizontal Pod Autoscaling**. 2024. Documentação do Kubernetes. Disponível em: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/. Acesso em: 14 out. 2026.

WILKES, J. **Google cluster-usage traces v3**. Mountain View, CA, USA, 2020. Disponível em: https://github.com/google/cluster-data.

XU, M. et al. esDNN: Deep neural network based multivariate workload prediction in cloud computing environments. **ACM Transactions on Internet Technology**, v. 22, n. 3, p. 75:1–75:24, 2022.